Informe de Solución a Prueba Técnica

Ingeniero de Desarrollo - Falabella Colombia

Luis Enrique Guerrero Ibarra.[1]

En respuesta a la prueba técnica presentada por **Ríos del Desierto SAS**, este proyecto aborda el desarrollo de una solución tecnológica para optimizar el Servicio de Atención al Cliente (SAC). El objetivo principal fue implementar una herramienta que permitiera consultar la información básica de un cliente mediante su número de documento, reducir los tiempos de atención y fomentar la fidelización de los clientes más valiosos.

La solución propuesta consta de dos aplicaciones integradas: un **Backend** desarrollado en **Python** utilizando **FastAPI** y **Flask**, y un **Frontend** basado en **Angular** con soporte para Progressive Web Applications (PWA). La arquitectura diseñada asegura una comunicación fluida entre ambos componentes a través de una API REST robusta y segura, respaldada por una base de datos **SQLite**.

En el Backend, se implementó una estructura modular y escalable, utilizando el **ORM SQLAlchemy** para el modelado de datos y **Pandas** para la generación de reportes automatizados. Los endpoints permiten realizar operaciones CRUD completas sobre clientes y productos, consultar información en tiempo real y generar reportes de fidelización en formato **Excel** para identificar a los clientes con mayores volúmenes de compra.

En el Frontend, se diseñó una interfaz funcional y dinámica que facilita la búsqueda y visualización de información de clientes, además de permitir la creación de nuevos registros cuando sea necesario. Se incorporaron funcionalidades clave como la exportación de datos en formato **CSV** y la generación de facturas, siguiendo buenas prácticas de usabilidad y accesibilidad.

El desarrollo incluyó una guía detallada de instalación y despliegue que asegura su correcta implementación en entornos productivos. La solución no solo cumple con los requerimientos establecidos, sino que también ofrece una base sólida para futuras ampliaciones, como la integración de analíticas avanzadas o técnicas de aprendizaje automático para optimizar la experiencia del cliente.

Este proyecto combina innovación tecnológica, diseño intuitivo y buenas prácticas de desarrollo para entregar una solución integral, eficiente y escalable que responde a las necesidades de **Ríos del Desierto SAS**. Este informe detalla cada etapa del desarrollo, la arquitectura empleada y los resultados obtenidos, posicionando a esta solución como una herramienta estratégica para la mejora del SAC.

---

[1] Desarrollador Web Full-Stack y Administrador Público

Introducción

La atención al cliente es un pilar fundamental para las empresas modernas, especialmente aquellas que operan en mercados competitivos donde la fidelización de los clientes representa una ventaja estratégica. En este contexto, **Ríos del Desierto SAS**, se propone como una empresa dedicada al comercio y servicios que, identificó la necesidad de optimizar los tiempos de respuesta en su Servicio de Atención al Cliente (SAC) y fortalecer la relación con sus clientes más leales.

La prueba técnica planteada por la empresa tenía como objetivo principal el desarrollo de una herramienta que permitiera consultar la información de un cliente mediante su número de documento, mostrando sus datos básicos y las compras asociadas. Además, la solución debía incluir funcionalidades clave como la generación de reportes de fidelización en formato **Excel** para identificar a los clientes con mayores volúmenes de compra, promoviendo estrategias dirigidas a fortalecer su lealtad hacia la marca.

Este proyecto aborda dicho desafío a través del desarrollo de dos aplicaciones interconectadas:

1. **Backend**: Una API desarrollada en **Python** utilizando **FastAPI** como framework principal, con soporte para operaciones CRUD y generación de reportes automatizados mediante **Pandas**.
2. **Frontend**: Una aplicación web **Angular**, con soporte para Progressive Web Applications (PWA), que proporciona una interfaz funcional y de fácil uso para el equipo de SAC.

El alcance de esta solución incluye:

- ✓ La creación de una arquitectura modular que garantice escalabilidad y mantenibilidad.
- ✓ El desarrollo de funcionalidades para la gestión de clientes, productos y compras.
- ✓ La implementación de endpoints seguros para la consulta y exportación de datos.
- ✓ La generación de reportes dinámicos que cumplen con los requerimientos específicos de la empresa.

En informe técnico se detallan las decisiones de diseño y desarrollo adoptadas, resaltando la importancia de las tecnologías utilizadas, las buenas prácticas aplicadas y el enfoque innovador empleado.

Diseño de la Arquitectura

Diagrama de Arquitectura

La solución se compone de dos módulos principales: el **Backend**, desarrollado en Python con **FastAPI** y **Flask**, y el **Frontend**, implementado en Angular con soporte para Progressive Web Applications (PWA). A continuación, se presenta un diagrama simplificado que ilustra la relación entre los componentes:

```
+--------------------+        HTTP API        +-------------------------+
|                    |------------------------>|                         |
|       Frontend     |                         |         Backend         |
| (Angular, PWA)     |<------------------------| (FastAPI, Flask, SQLite)|
|                    |        Respuestas       |                         |
+--------------------+                         +-------------------------+
          |                                              |
          +-----------------> Base de Datos SQLite <-------+
                                (Gestión de Clientes,
                                Compras y Productos)
```

Descripción de los Módulos

Backend

El módulo Backend se centra en la lógica de negocio, la persistencia de datos y la generación de reportes. Está diseñado con una estructura modular y organizada, destacando los siguientes aspectos:

Modelado de datos (ORM)

Utiliza **SQLAlchemy** para definir tablas como Cliente, Compra, Producto y TipoDocumento.

Gestión de Clientes

Incluye operaciones CRUD para la tabla Cliente (creación, actualización, consulta y eliminación) mediante endpoints en **FastAPI**.

Gestión de Productos

Permite administrar productos, sus precios y atributos específicos a través de endpoints REST.

Reportes de Fidelización

Genera archivos en formato Excel con los clientes que superan un monto específico de compras, implementado con **Pandas** para manipulación de datos y **FastAPI** para la descarga.

Exportación de Clientes

Proporciona archivos CSV con información completa de los clientes.

Flask Admin

Interfaz administrativa para gestionar datos desde un navegador. Incluye páginas HTML personalizadas para clientes, productos y compras.

Frontend

El módulo Frontend ofrece una interfaz de usuario funcional para facilitar la interacción con el sistema. Algunos aspectos destacados son:

Estructura Modular

Organizado en componentes reutilizables (navbar, footer, product-list, etc.) y funcionalidades (client-form, home, etc.).

Conexión con API

Realiza solicitudes HTTP al Backend utilizando servicios Angular para obtener datos de clientes y productos.

Exportación y Reportes

Permite exportar datos de clientes y generar reportes directamente desde la interfaz.

PWA

La integración como aplicación web progresiva facilita su instalación en dispositivos y mejora la experiencia del usuario en modo offline.

Integraciones y Comunicaciones

API REST

El Frontend se comunica con el Backend mediante solicitudes HTTP (GET, POST, PUT, DELETE), consumiendo endpoints desarrollados en FastAPI.

Base de Datos SQLite

Base de datos ligera que almacena información persistente sobre clientes, productos y compras.

Librerías de Soporte

Backend: Pandas (manipulación de datos), SQLAlchemy (ORM), FastAPI (API REST), Flask (administración).

Frontend: Angular Material (diseño de interfaz), jsPDF (generación de PDFs).

Justificación de Decisiones Tecnológicas

FastAPI

Seleccionado por su alto rendimiento y facilidad para desarrollar APIs modernas con validación de datos integrada y un rápido desarrollo.

Flask

Para crear una interfaz administrativa, ligera y extensible.

Angular

Estructura modular y soporte para PWA adecuado, para desarrollar aplicaciones escalables y dinámicas.

SQLite

Para entornos ligeros donde no se requiere una base de datos compleja.

Esta arquitectura asegura una solución escalable, eficiente y fácilmente mantenible, cumpliendo con los requerimientos del cliente.

Detalles del Backend: Estructura del Proyecto

El proyecto Backend está organizado de manera modular para maximizar la mantenibilidad, escalabilidad y legibilidad. A continuación, se describe la estructura principal:

Assets > Scripts > arquitecturaBackend.gql

```
SAC-Optimizer/
├── app/
│   ├── __init__.py
│   ├── main.py                 # Punto de entrada de la API FastAPI
│   ├── controllers/            # Endpoints principales
│   │   ├── clientes.py         # Gestión CRUD de clientes
│   │   ├── productos.py        # Gestión CRUD de productos
│   │   ├── tipo_documento.py   # Gestión CRUD de tipos de documento
│   │   ├── ventas.py           # Registro de ventas
│   │   ├── reporte_fidelizacion.py # Generación de reportes de fidelización
│   │   ├── exportar_clientes.py    # Exportación de datos en formato CSV
│   │   └── flask_admin.py      # Panel administrativo con Flask
│   ├── database/               # Configuración y modelos de base de datos
│   │   ├── config.py           # Configuración de la conexión con SQLite
│   │   ├── models.py           # Modelado de datos con SQLAlchemy
│   │   └── sac_optimizer.db    # Archivo de la base de datos SQLite
│   ├── schemas/                # Esquemas de validación y respuesta
│   ├── services/               # Lógica de negocio y utilidades
│   │   ├── export_to_csv.py    # Exportación de datos a CSV
│   │   └── report_service.py   # Generación de reportes en Excel
│   ├── templates/              # Plantillas HTML para Flask Admin
│   └── tests/                  # Pruebas unitarias y de integración
```

Modelo de Datos (ORM)

El modelo de datos se define utilizando **SQLAlchemy**, organizando las entidades principales:

Cliente: Representa a los clientes del sistema. Se relaciona con TipoDocumento, Compra e Historial.

Compra: Registra las compras realizadas por los clientes, relacionadas con productos específicos y vinculadas a la tabla Cliente.

Producto: Contiene información sobre los productos disponibles para la venta, como precio, talla y color.

TipoDocumento: Tabla de referencia para los tipos de documento (ej., CC, NIT, TI).

Historial: Registra las interacciones y eventos relacionados con los clientes.

Las relaciones están diseñadas con integridad referencial, utilizando claves foráneas y cascadas para facilitar la gestión de los datos.

Endpoints Principales

1. **Gestión de Clientes:** CRUD completo en clientes.py.

   POST /clientes: Crea un nuevo cliente.

   GET /clientes: Lista todos los clientes.

   PUT /clientes/{id}: Actualiza los datos de un cliente.

   DELETE /clientes/{id}: Elimina un cliente.

2. **Gestión de Productos:** CRUD implementado en productos.py.

   POST /productos: Crea un producto.

   GET /productos: Lista productos con soporte para paginación.

   PUT /productos/{id}: Actualiza un producto.

   DELETE /productos/{id}: Elimina un producto.

3. **Reportes de Fidelización:** En reporte_fidelizacion.py.

   GET /reporte_fidelizacion: Genera un reporte en Excel de los clientes más valiosos.

4. **Exportación de Clientes:** Implementado en exportar_clientes.py.

   GET /exportar/clientes: Exporta datos de clientes a un archivo CSV.

5. **Panel Administrativo (Flask):** Permite gestionar datos desde una interfaz web sencilla.

Uso de Librerías Especiales

Pandas: Utilizado en report_service.py para generar reportes dinámicos en Excel con datos filtrados de la base de datos.

CSV: Implementado en export_to_csv.py para exportar información básica y resumida de los clientes.

SQLAlchemy: Base del modelado de datos y consultas en la base de datos.

Exportación de Datos

CSV (Exportación de Clientes):

Genera un archivo con información detallada de los clientes, incluyendo total de compras y fecha de última compra.

Excel (Reporte de Fidelización):

Crea un reporte en Excel con los clientes que superan las metas de compra en el último mes. Se aplica un filtro SQL para calcular el total de compras y se utiliza Pandas para generar el archivo.

Buenas Prácticas Aplicadas

Documentación: Cada endpoint está documentado con descripciones claras en su implementación.

Código Modular y Escalable: Separación de responsabilidades entre controladores, servicios y modelos.

Pruebas Unitarias y de Integración: Se incluyen tests para validar la funcionalidad de los endpoints y el correcto funcionamiento de las operaciones CRUD.

Manejo de Excepciones: Implementación de control de errores detallado con respuestas HTTP apropiadas para evitar fallos silenciosos.

Organización del Código: Estructura clara y comentada que facilita la lectura y el mantenimiento.

Detalles del Frontend

Estructura del Proyecto

El Frontend fue desarrollado utilizando **Angular** con soporte para **Progressive Web Applications (PWA)**. La estructura modular permite mantener el código organizado, facilitando la escalabilidad y el mantenimiento.

```
Assets > Scripts > arquitecturaFrontend.gql

LuceModa
│
src/
├── app/
│   ├── app.component.*          # Componente principal
│   ├── components/              # Componentes reutilizables (navbar, footer, etc.)
│   ├── features/                # Funcionalidades principales (gestión de clientes, productos, etc.)
│   ├── models/                  # Modelos de datos utilizados en la aplicación
│   ├── services/                # Lógica de negocio y conexión con la API
│   ├── shared/                  # Módulos compartidos como Angular Material
│   └── environments/            # Configuraciones de entorno (desarrollo, producción)
```

Estructura principal

Componente Principal: HomeComponent

Funcionalidad

Proporciona una interfaz para buscar clientes por tipo y número de documento.

Permite navegar a la selección de productos o registrar un cliente nuevo.

Incluye botones para exportar datos de clientes y generar reportes de fidelización.

Formulario de Búsqueda

Implementado con ReactiveFormsModule para validaciones en tiempo real. Los campos incluyen:

Tipo de Documento: Selector desplegable con opciones cargadas desde el servicio.

Número de Documento: Campo de texto para ingreso manual.

Resultado de Búsqueda

- ✓ Muestra la información del cliente si es encontrado.
- ✓ Opción de registrar al cliente si no existe en la base de datos.
- ✓ Navegación hacia la selección de productos con datos del cliente.

Funciones Adicionales

- ✓ Exportación de clientes en formato CSV.
- ✓ Generación de reportes de fidelización en Excel, consumiendo endpoints del Backend.

Archivo home.component.html

El diseño combina la simplicidad y funcionalidad, destacando el uso de **Angular Material**.

Formulario: Estructurado con mat-form-field y validaciones requeridas.

Resultados de búsqueda: Bloque condicional que muestra la información del cliente o sugiere registrarlo.

Botones adicionales:

Exportar clientes.

Generar reporte de fidelización, con una insignia que muestra el número de clientes elegibles.

Archivo main.ts

Configura el inicio de la aplicación:

Service Worker: Registrado en modo producción para mejorar la experiencia offline y las capacidades PWA.

Bootstrap Application: Inicia el AppComponent como raíz de la aplicación.

Puntos Clave

Interacción con la API

Servicios como ClientsService manejan las conexiones con los endpoints del Backend.

Operaciones principales:

Buscar clientes (GET /clientes).

Exportar datos (GET /exportar/clientes).

Generar reportes de fidelización (GET /reporte_fidelizacion).

Buenas Prácticas

Uso de **Angular Material** para garantizar consistencia en el diseño y accesibilidad.

Modularización del código (features/, services/).

Validaciones en el formulario de búsqueda, asegurando entradas válidas antes de realizar peticiones.

Exportación y Reportes

Los botones para exportar y generar reportes interactúan dinámicamente con el Backend.

Descargas gestionadas mediante blobs para garantizar compatibilidad en navegadores.

Progresividad y Accesibilidad

La integración de PWA mejora la disponibilidad de la aplicación en redes inestables.

Uso de prácticas modernas como ReactiveFormsModule y HttpClientModule.

Innovación y Uso de Ciencia de Datos

Aplicaciones de Ciencia de Datos en el Proyecto

Aunque el objetivo principal del proyecto no estaba directamente relacionado con el análisis predictivo o el aprendizaje automático, se implementaron técnicas de manipulación y análisis de datos que se alinean con los principios básicos de la ciencia de datos, maximizando la eficiencia y la utilidad del sistema.

Generación de Reportes de Fidelización

Lógica Aplicada:

El sistema utiliza **Pandas**, una librería ampliamente reconocida en ciencia de datos, para procesar y analizar los datos de compras de los clientes.

Criterios de Fidelización:

Se seleccionan clientes con compras superiores a $5,000,000 COP en el último mes.

Se consolidan los datos en un archivo Excel estructurado para su análisis.

Impacto:

Este enfoque permite a la empresa priorizar esfuerzos de fidelización en clientes de alto valor, optimizando sus recursos y aumentando su tasa de retención.

Exportación de Datos

Manejo de Datos:

Se genera un archivo **CSV** con los datos de los clientes, incluyendo su historial de compras y otras métricas relevantes.

Automatización y Flexibilidad:

La estructura del archivo facilita su integración con otras herramientas de análisis.

Incluye métricas clave como el total de compras y la última fecha de interacción.

Visualización y Accesibilidad de Datos

Frontend:

El uso de insignias (mat-badge) para mostrar la cantidad de clientes elegibles para fidelización brinda una visualización clara y directa de los datos en tiempo real.

Backend:

La generación de reportes y la exportación de datos están diseñadas para ejecutarse en tiempo real, ofreciendo resultados inmediatos y accionables.

Oportunidades para Extender el Uso de Ciencia de Datos

El sistema actual establece una base sólida que puede expandirse para incluir técnicas más avanzadas de ciencia de datos en el futuro:

Análisis Predictivo

Usar modelos de machine learning para predecir comportamientos de compra futuros y segmentar a los clientes en grupos de interés.

Visualización Avanzada

Implementar dashboards interactivos para visualizar métricas clave, como tendencias de compras o análisis geográfico de los clientes.

Optimización de Recomendaciones

Integrar un sistema de recomendaciones basado en historial de compras y productos relacionados.

Detección de Anomalías

Utilizar algoritmos de detección de anomalías para identificar posibles errores en los datos de ventas o comportamientos inesperados en las compras.

Validación de Datos con Schemas

Uso de Pydantic para la Validación de Datos

En el proyecto, **Pydantic** se empleó para definir y validar los datos en las interacciones con la API, asegurando que solo se acepten entradas válidas y consistentes antes de realizar operaciones en la base de datos. Cada esquema describe la estructura de los datos, sus tipos y las restricciones asociadas.

Principales Ventajas de Pydantic

Validación Automática: Los datos enviados al Backend se validan automáticamente contra los esquemas definidos.

Flexibilidad: Soporte para transformaciones automáticas (ej., convertir cadenas a fechas).

Errores Detallados: Respuestas claras en caso de que los datos no cumplan con los requisitos.

Ejemplos de Schemas Implementados

Cliente

- ✓ Define los datos básicos requeridos para gestionar un cliente.
- ✓ Campos obligatorios como tipo_documento, numero_documento, nombre y correo.

- ✓ Validación automática de correos únicos.
- ✓ Estos esquemas se utilizan al crear o actualizar clientes, asegurando que la información sea válida y completa.

Producto

- ✓ Se asegura de que los productos tengan precios válidos y atributos opcionales como talla y color.

Compra

- ✓ Incluye validaciones para garantizar que las compras estén asociadas a un cliente existente.
- ✓ Calcula automáticamente el monto total según la cantidad y precio del producto.

Integración con Endpoints

- ✓ Los esquemas se integran directamente con los controladores de FastAPI, simplificando el manejo de datos y validando automáticamente los parámetros de entrada:
- ✓ Este método asegura que solo se procesen solicitudes con datos válidos, rechazando automáticamente aquellas que no cumplan los requisitos del esquema.

Buenas Prácticas en la Implementación de Schemas

Separación de Responsabilidades

Los schemas se encuentran en su propio módulo (schemas/) para mantener la claridad del código y facilitar su mantenimiento.

Validación Consistente

Se utilizan tanto para entrada como para salida de datos, garantizando uniformidad en todas las operaciones.

Transformaciones y Restricciones

Uso de validadores avanzados en Pydantic para convertir formatos o limitar valores según los requisitos del negocio.

Documentación Automática

Al integrarse con FastAPI, los esquemas generan documentación interactiva en /docs, detallando los campos requeridos y sus tipos.

Esta implementación robusta asegura la calidad de los datos, protege la integridad de la base de datos y reduce errores en las interacciones del sistema.

Despliegue y Guía de Implementación

Requerimientos del Entorno

Antes de iniciar el despliegue, es importante asegurarse de cumplir con los siguientes requisitos:

Backend

Python: Versión 3.9 o superior.

Dependencias: Administradas mediante pip, instaladas desde requirements.txt.

Base de Datos: SQLite ya configurada en el archivo sac_optimizer.db.

Frontend

Node.js: Versión 14 o superior.

Angular CLI: Versión 12 o superior.

Entorno de Navegador: Para PWA, soporte para **Service Workers**.

Herramientas Recomendadas

Git: Para la clonación y administración del repositorio.

VS Code: Para la edición de código y configuración del proyecto.

Pasos de Implementación

Clonación del Repositorio:

Clona el repositorio del proyecto en tu entorno local:

```
git clone https://github.com/usuario/SAC-Optimizer.git
cd SAC-Optimizer
```

Configuración del Backend

Crear un Entorno Virtual:

```
python -m venv env
source env/bin/activate
# En Windows: env\Scripts\activate
```

Instalar Dependencias:

```
pip install -r requirements.txt
```

Configurar la Base de Datos:

La base de datos SQLite (sac_optimizer.db) está precargada y lista para su uso.

Si es necesario, ejecuta el script de configuración:

```
python app/database/setup.py
```

Iniciar el Servidor:

```
uvicorn app.main:app --reload
```

- ✓ Verificar Documentación de la API: Accede a http://127.0.0.1:8000/docs para explorar los endpoints disponibles.

Configuración del Frontend

- ✓ Instalar Dependencias: Dirígete al directorio del Frontend y ejecuta:

```
npm install
```

- ✓ Iniciar en Modo Desarrollo:

```
ng serve
```

Accede a http://localhost:4200/ para ver la aplicación en acción.

- ✓ Construir para Producción:

```
ng build --configuration production
```

- ✓ Despliegue del Frontend:

Sube los archivos compilados a un servidor web (como **Netlify**, **Firebase Hosting**, o un servidor Nginx).

Configura el servidor para que sirva el archivo index.html en todas las rutas.

Integración del Panel Administrativo (Flask)

- ✓ Iniciar la Aplicación Flask:

```
python app/controllers/flask_admin.py
```

Accede a http://127.0.0.1:5000/admin para gestionar datos desde la interfaz administrativa.

- ✓ Verificar el Panel:

Gestionar clientes, productos y compras directamente desde la interfaz.

Consideraciones Finales para el Despliegue

- ✓ **Variables de Entorno**: Configura valores sensibles (como credenciales de acceso) en variables de entorno para evitar exponerlos en el código.

Seguridad:

- ✓ Usa HTTPS para todas las conexiones entre Frontend y Backend.
- ✓ Configura CORS adecuadamente para evitar accesos no autorizados.

Supervisión y Logs:

- ✓ Implementa un sistema de monitoreo como Prometheus o Sentry.
- ✓ Revisa periódicamente los logs del servidor y el navegador.

Mantenimiento de la Base de Datos:

- ✓ Realiza respaldos periódicos de la base de datos SQLite.
- ✓ Evalúa migrar a un sistema más robusto como PostgreSQL en caso de que la carga aumente.

Esta manera asegura que el sistema pueda ser configurado y desplegado sin contratiempos, manteniendo la calidad y disponibilidad esperadas.

Puntos de Acceso Clave:

Backend, accede a la gestión de la base de datos a través del archivo **app/database/config.py.**

Frontend, accede a la gestión de la API Rest a través del archivo: **src/app/services/api.service.ts**

Conclusiones

El desarrollo de este proyecto ha permitido crear una solución tecnológica robusta y escalable para optimizar el Servicio de Atención al Cliente (SAC) de **Ríos del Desierto SAS**. Al combinar un **Backend** eficiente desarrollado en Python y un **Frontend** moderno con Angular, la solución cumple con los requerimientos establecidos, proporcionando herramientas clave para la gestión de clientes y productos, así como para la generación de reportes estratégicos.

Puntos destacados del proyecto

Cumplimiento de los Requerimientos:

Funcionalidades completas de búsqueda, consulta y registro de clientes mediante un formulario intuitivo.

Generación de reportes de fidelización en formato **Excel** y exportación de datos en **CSV**.

Gestión eficiente de la base de datos mediante **SQLAlchemy** y validación con **Pydantic**.

Uso de Buenas Prácticas:

Arquitectura modular que asegura escalabilidad y mantenibilidad.

Validación rigurosa de los datos de entrada y salida.

Integración de un panel administrativo con Flask para la gestión interna.

Impacto Empresarial:

Reducción de tiempos en las llamadas de soporte al cliente.

Identificación de clientes de alto valor para estrategias de fidelización.

Optimización del flujo de trabajo del equipo SAC.

Documentación:

Cada aplicación contiene un archivo **README.md** con la **Guía completa de Implementación** con su paso a paso para la descarga, instalación y ejecución de las aplicaciones, tanto **Backend: SAC-Optimizer**, como **Frontend: LuceModa.**

Repositorios en GitHub de las aplicaciones:

Url Backend de la aplicación **SAC-Optimizer**:

https://github.com/LuisEnGuerrero/SAC-Optimizer.git

Url Frontend de la aplicación **LuceModa**:

https://github.com/LuisEnGuerrero/LuceModa.git

Actualizaciones Futuras

El sistema desarrollado sienta las bases para futuras mejoras que podrían aumentar aún más su utilidad y alcance.

1. Mejoras en la Experiencia del Usuario (UX/UI):

- ✓ Implementar un diseño más atractivo e intuitivo en el Frontend, incorporando dashboards visuales para reportes.

2. Ampliación del Sistema:

- ✓ Agregar más tablas relacionadas, como Historial de Contactos u Ofertas de Fidelización.
- ✓ Creación de Componentes adicionales para abrir una Tienda OnLine.

3. Implementación de Técnicas Avanzadas de Ciencia de Datos:

- ✓ Introducir un sistema de recomendaciones para productos basados en el historial de compras de los clientes.
- ✓ Utilizar algoritmos de clustering para segmentar clientes y definir estrategias personalizadas de fidelización.

4. Optimización del Backend:

- ✓ Migrar a una base de datos más abierta como MongoDb para desplegar la aplicación a producción sin acarrear un gran costo.
- ✓ Migrar a una base de datos más robusta como PostgreSQL para manejar volúmenes de datos más grandes.

5. Pruebas Automatizadas y Mejora Continua:

- ✓ Implementar un conjunto completo de pruebas unitarias e integradas para garantizar la estabilidad del sistema.
- ✓ Establecer pipelines de integración y entrega continua (CI/CD) para acelerar las actualizaciones y correcciones.

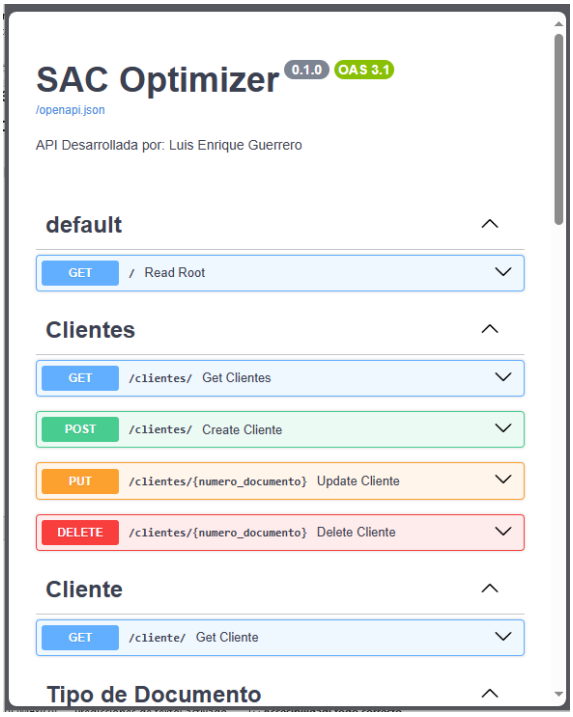
6. Seguridad y Auditoría:

- ✓ Añadir autenticación y autorización para proteger los datos y limitar el acceso según roles.
- ✓ Implementar auditorías automáticas para rastrear cambios en la base de datos.

Anexos

Presentación de SAC-Optimizer:

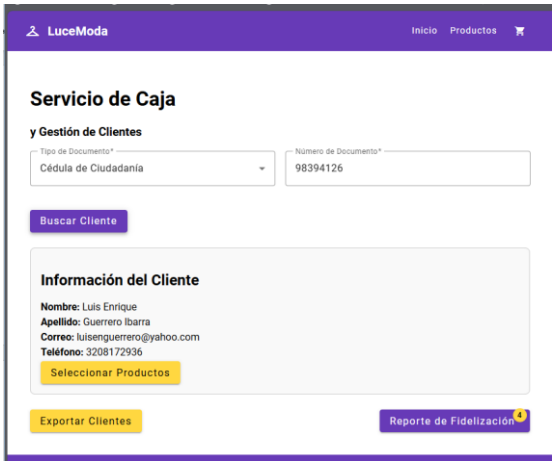
Módulo de documentación: http:localhost:8000/Docs



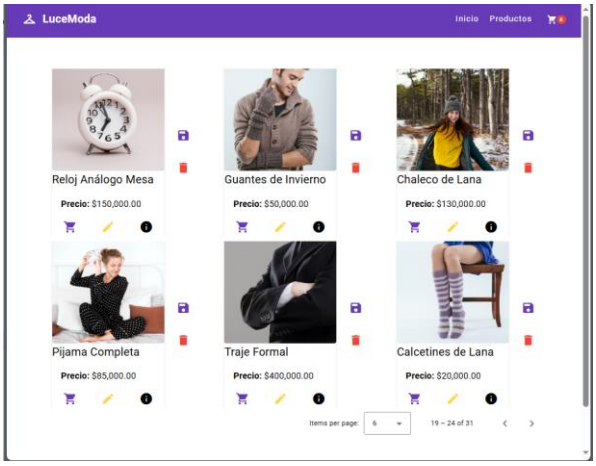
Módulo de Administración con Flask: http:localhost:8000/admin

Presentación de LuceModa:

Página de Inicio de la Aplicación http://localhost:4200/



Página de Selección de Productos http://localhost:4200/products



Créditos:

Luis Enrique Guerrero Ibarra
Desarrollador Web FullStack
LuisEnGuerrero@yahoo.com
https://LuisEnGuerrero.Netlify.App